

Du Xujia

AI Application Engineer | RAG | Agent Runtime | Backend Engineering

☎ +86 150 3592 5107 ✉ 2041487752dxj@gmail.com 🌐 GitHub 📁 Portfolio 🌐 Global Site

Target roles: AI Application Engineer | LLM Application Engineer | Backend Engineer, AI Systems

Profile

AI application and backend engineer focused on RAG systems, Agent Runtime, business backends, and multi-platform products. I connect AI capabilities to real operational workflows instead of leaving them as single-endpoint demos. Comfortable with Python/FastAPI, Java/Spring Boot, and React/Next.js, with hands-on ownership from solution design to implementation, integration, testing, and release.

I care about traceability, recovery, replayability, and verification: retrieval results should expose evidence, async tasks should recover after failure, payment/device callbacks should be idempotent, and critical paths should be guarded by tests and runtime metrics.

Core Strengths

- **AI application engineering:** RAG, LangGraph, tool calling, workflow orchestration, multi-channel access, and knowledge-base governance; able to connect Q&A flows to business actions such as lookup, submission, and human handoff
- **Backend system design:** authentication, payment, task workflows, admin systems, data migration, and performance tuning with FastAPI, Spring Boot, PostgreSQL, Redis, and ClickHouse
- **Quality and release discipline:** pytest, Vitest, Playwright, lint/build, health checks, regression gates, and rollback plans for release risk control
- **Verified outcomes:** optimized a China Unicom analytics API from **20s+ to 4s**, migrated **300+ tables and 300M+ records**, and reduced selected core queries from **10s+ to 500ms**

Work Experience

Independent Developer

2025.12 - Present

AI Applications | Full-stack Products

RentBox Shared Window-cleaning Device Mini Program

Spring Boot | React | uni-app

- **Payment-device consistency:** designed an order lifecycle service and state machine for a shared-cabinet rental flow where payment, deposit-free authorization, pickup, return, and device callbacks can arrive out of order; used row locks, state history, and audit logs to keep unlock eligibility based on server-side truth.
- **Idempotent callbacks and compensation:** integrated WeChat Pay v3, WeChat Pay Score, and cabinet Web APIs; handled duplicate notifications, timeout retries, and manual compensation through Redis/JDBC idempotency, distributed locks, and callback replay to reduce duplicate charges, wrong unlocks, and state rollback risk.
- **Multi-end delivery gates:** coordinated `server / admin / channel / miniapp`, consolidating `231` Java source files and `100` test files around payment, device, and order flows; used `mvn -q test`, frontend builds, and mini-program regression as release gates.

Paper Retrieval Task Platform

FastAPI | React | PostgreSQL

- **Shared-table task boundary:** built user-side task creation, status sync, result download, and quota display while the client's Worker only read/wrote a shared PostgreSQL `tasks` table and could not call back into the user system.
- **Terminal-state contract and quota safety:** moved quota deduction after successful task completion; constrained terminal rollback and success fields through Alembic check constraints/triggers, then used `SELECT ... FOR UPDATE`, unique quota events, and concurrency tests to prevent duplicate deductions.
- **Engineering fallback for integration risk:** hardened the task system with `15` Alembic migrations, `14` API test files, and `20` frontend/E2E test files, continuously checking Worker write-back contracts, quota ledger behavior, and SSE fallback paths.

AI Customer Service Runtime

Agent Runtime | FastAPI

- **Business-oriented customer service runtime:** expanded support from single-turn RAG Q&A into a runtime that accepts text, voice/RTC, host-page context, and business lookups, with clear boundaries for channels, host bridge, core engine, business tools, plugins, and model adapters.
- **Routing and host decoupling:** routed by ``intent_stack``, ``page_context``, and ``business_objects``, sending low-confidence or high-risk cases to human handoff; reused host authorization through seven plugin types and an AuthBridge instead of hard-coding industry tools into the main chain.
- **Plugin-based runtime skeleton:** unified text, Voice API, RTC WebSocket, and FastAPI sub-app access patterns; extracted Route, BusinessTool, Handoff, Industry, AuthBridge, Context, and Response extension points for multi-tenant customer service integration.

WeChat Intelligent Assistant

LangGraph | Electron | RAG

- **Desktop automation stability:** abstracted a ``BaseTransport`` layer to absorb differences across Windows WeChat versions, permissions, and unstable message channels; used LangGraph to decouple first response from background growth tasks such as fact extraction, vector write-back, and profile updates.
- **Long-running operations:** built SQLite short-term memory, runtime vector memory, and exported-corpus RAG within a 2-second reply budget; added ``/api/status``, ``/api/metrics``, config audit, backup/restore, and offline eval so failure diagnosis and quality regression are verifiable.
- **Three-layer memory and ops surface:** evolved the local Agent into a three-layer memory stack with lightweight reranking and local ``Cross-Encoder`` fallback, plus cost analysis, config audit, and backup/restore for long-term operation.

Chinasoft International

2025.04 - 2025.09

Enterprise Knowledge-base RAG | Agent Workbench | Backend/Full-stack Engineer

- **Evidence-first RAG:** designed structured, full-text, and vector retrieval paths for enterprise knowledge bases where answers must explain evidence, refusal boundaries, and retrieval diagnostics; combined query rewrite, weighted RRF, and rerank to return citations, ``grounding_score``, and ``trace_id``.
- **Recoverable runtime and regression gates:** migrated the Gateway Q&A flow to LangGraph with checkpoint, interrupt/resume, and human clarification; built ingest, knowledge governance, retrieve/debug, prompt-injection protection, smoke eval, and regression gates to make answer quality part of release validation.
- **Knowledge governance workbench:** exposed multi-knowledge-base management, multi-source connectors, chunk split/merge/disable, and retrieval debugging, reducing manual dependence for version switching, citation alignment, and refusal validation.

National Clinical Research Center for Orthopedics

2024.08 - 2024.10

SubMed Medical Literature Search Mini Program | Backend Developer Intern

- **Medical search loop:** participated in API design and integration for a mini program that helps doctors search PubMed literature by disease, surgery type, and research direction, covering AI search, voice input, paper detail, recommendation, favorites, deep analysis, and sharing.
- **Continuous research feed:** integrated ``/api/articles/ai_search``, ``/api/deep_analysis``, ``/api/recommend_paper``, subscription settings, and push history, turning one-time searches into trackable research feeds by keyword, impact factor, and Chinese Academy of Sciences partition.
- **Post-search analysis and recommendation:** connected WeChat speech translation input, paper-detail enhancements, favorites, and recommendation flows so search results could continue into analysis, review, and research-topic subscription scenarios.

Touchpoint Enablement Center | Shaanxi Operations Platform | Data Platform | Backend Developer Intern

- **Touchpoint enablement center:** rebuilt external API secondary authorization and separated OLTP/OLAP workloads with ClickHouse aggregation, reducing an activity statistics API from **20s+ to 4s** for high-frequency operations reporting.
- **Shaanxi operations platform:** developed capability management pages and backend logic, handled request/response encryption and decryption, maintained number reactivation APIs, and implemented model-based number scanning plus automatic activation to support operations automation.
- **Data platform migration:** migrated **300+ tables and 300M+ records** without interrupting business, verified consistency, and used xxl-job for daily dynamic updates; rewrote aggregation SQL and added covering indexes/caches, reducing selected core queries from **10s+ to 500ms** and release time from **30 minutes to 5 minutes**.

| Projects

RAG-QA System

FastAPI | Vue | LangGraph | Qdrant

- **Multi-knowledge-base Q&A:** built for enterprise knowledge-base scenarios where answers must trace back to sources, covering document upload, async parsing, unified chat, SSE streaming answers, citation return, and retrieval diagnostics.
- **Three-path retrieval and citation grounding:** implemented structured, full-text, and vector retrieval in the retrieval service; combined query rewrite, RRF, and rerank to return citations, `grounding\_score`, latency breakdown, and `trace\_id`.
- **LangGraph flow refactor:** refactored Q&A into LangGraph with checkpoint, interrupt/resume, and `step\_events`, reducing repeated retrieval and duplicate cost for the same question.

EasyCloudPan

Spring Boot | React | PostgreSQL | Redis | MinIO

- **Chunked upload and resumable transfer:** designed instant upload, resumable upload, and SSE progress reporting for large files, weak networks, and duplicate uploads; supported **1000+ concurrent uploads** and **>99.5%** upload success rate.
- **Request signing and security boundaries:** designed HMAC-SHA256 + timestamp + nonce request signing, JWT dual tokens, magic-number validation, multi-tenant isolation, and file-access aspects to tighten platform security.
- **Multi-level cache and monitoring:** introduced Caffeine/Redis/DB multi-level caching plus Prometheus/Grafana monitoring; documented API P95 **<500ms** and database query P95 **<100ms**.

IPA Dialect Speech Transcription Prototype

Flask | PyTorch | Transformers | torchaudio

- **IPA dialect speech transcription:** delivered a local Web prototype for organizing IPA transcription after dialect speech collection, supporting batch upload, recording, single-file retranscription, SSE progress, and Excel export.
- **Audio normalization and phonetic splitting:** used `transformers.pipeline`, `torchaudio`, and `ffmpeg` to standardize audio input, then split IPA output into initials, finals, and tones for researcher review and cleanup.
- **Batch processing and review flow:** used local model `quanliang/checkpoint-133290`, `ThreadPoolExecutor`, and recognition caching for batch processing; repository evidence includes `250` WAV files and `6` XLSX templates/rule files.

Jointown Four-way Shuttle Path Planning System

Python | A* | PyQt5 | multiprocessing

- **Four-way shuttle path planning:** modeled a 42x27 warehouse grid for roads, shelves, and obstacles in pharmaceutical logistics; built A* pathfinding, map editing, and multi-vehicle path visualization prototypes.
- **Traffic rules and conflict handling:** designed separate passability rules for empty and loaded vehicles, plus priority-based timestep conflict handling; used PyQt5 to display search steps, waiting actions, and scheduling results frame by frame.
- **Multi-version experiments:** iterated across `cbs.py`, `test5-5.py`, `three.py`, `final.py`, and other Python experiment scripts, evolving from single-vehicle pathfinding to multi-vehicle, process-pool, and conflict-resolution validation.

| Skills

- **AI engineering:** RAG, LangGraph, Agent Runtime, tool calling, workflow orchestration, OpenAI API, vector retrieval, citation grounding
- **Backend and data:** Python, FastAPI, AsyncIO, Pydantic, SQLAlchemy, Java, Spring Boot, MyBatis, PostgreSQL, MySQL, Redis, ClickHouse
- **Full-stack delivery:** React, Next.js, TypeScript, Vite, uni-app, REST APIs, payment integration, admin systems, multi-end coordination
- **Engineering quality:** pytest, pytest-asyncio, Vitest, Playwright, Prometheus, Grafana, health checks, rollback, lint, build, E2E

| Education

Nanyang Institute of Technology | Data Science and Big Data Technology

2021.09 - 2025.06

- GPA 3.8/4.5, Top 5%
- University Merit Student, Outstanding Student Cadre, School-level Scholarship